01 / THE PROBLEM & OUR VISION

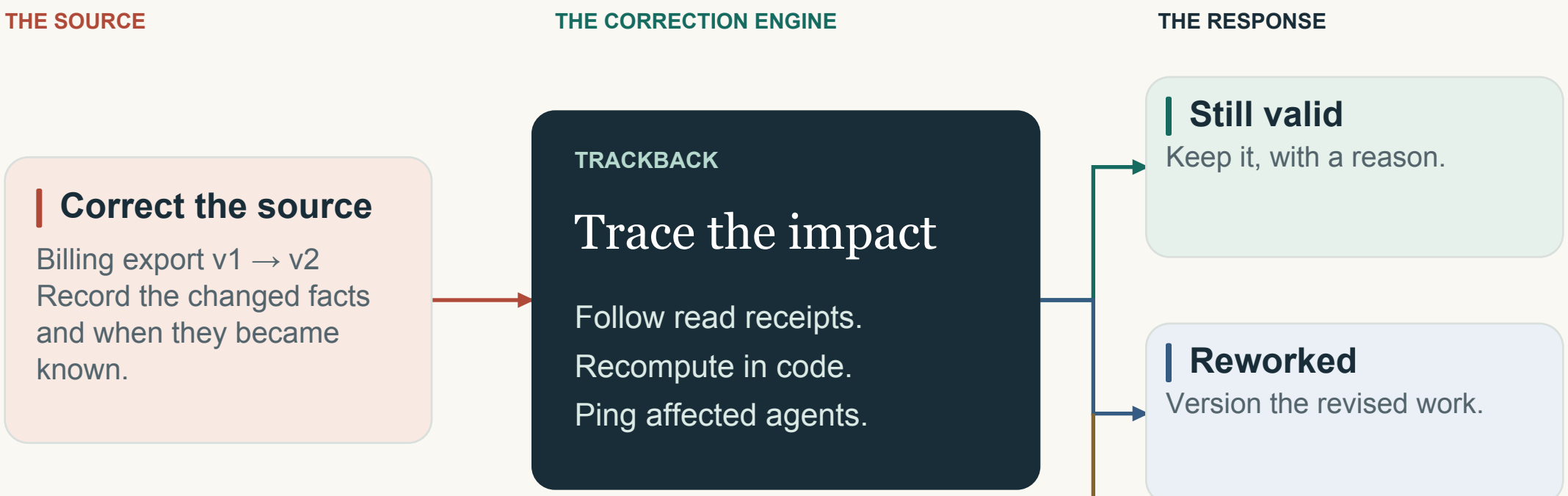
One correction. Many consequences.

Finance agents build on each other's work. When an input changes later, the team needs to know what broke, what still holds, and who must act.

FICTIONAL CASE STUDY · TIDEWALL'S SIX GYMS

A billing duplicate reaches beyond the ledger.

A $118,000 March revenue correction can change a lender certificate, a board deck, bonuses and a tax estimate. Finding every dependency by hand is the work Trackback targets.



Central insight

Record what each output actually consumed, then trace corrections forward.

THE DEMO'S TURNING POINT

2.95× → 3.03×

The leverage calculation crosses a 3.00× covenant limit.
A small-looking revenue error becomes a consequential decision.

BUILT FOR

Close accountants & reporting analysts: targeted rework.
Controllers & CFOs: evidence, approval and an auditable correction trail.

YOUR REVIEW Circle the downstream consequence that matters most to a finance team.

EVIDENCE README · product Live E11 flow Recorded decisions

02 / IDEATION & EXPLORATION

Why selective repair?

Retrospective map: reconstructed from the implementation and decision log.
This records the reasoning visible in the repository, not an earlier brainstorming session.

01 Problem

Late corrections
Cross-agent dependencies
Actions already executed

02 Trust

Exact versions consumed
No future-data leakage
Human approval for fixes

THE DESIGN QUESTION

What actually needs to change?

03 Alternatives

Rerun every agent: expensive.
Flag every descendant: noisy.
Both exist as evaluation baselines.

04 Implemented

Recompute first.
Check relevance and materiality.
Bundle suspect work into one ping per agent per wave.

05 Evidence

Two clocks: effective time and known time.
Pinned reads preserve what an agent could know.

06 Control

Receipts verify dependencies.
Caps bound rework.
Compensations repair actions that already happened.

THREE CHOICES THAT SHAPE THE BUILD

01	**Precision over blanket reruns**	Identical values stop the walk; below-threshold changes still flow to children, preserving aggregate effects.
02	**Reproducibility over live dependence**	Record, replay and scripted modes keep the demo usable offline; the snapshot contains mixed response origins.
03	**Verified scope over a rewrite**	The Oct 24 decision kept the six-gym world and added Jade + lessons instead of replacing it with the 48-gym proposal.

YOUR REVIEW Highlight one tradeoff to revisit. Add the alternative and the evidence needed.

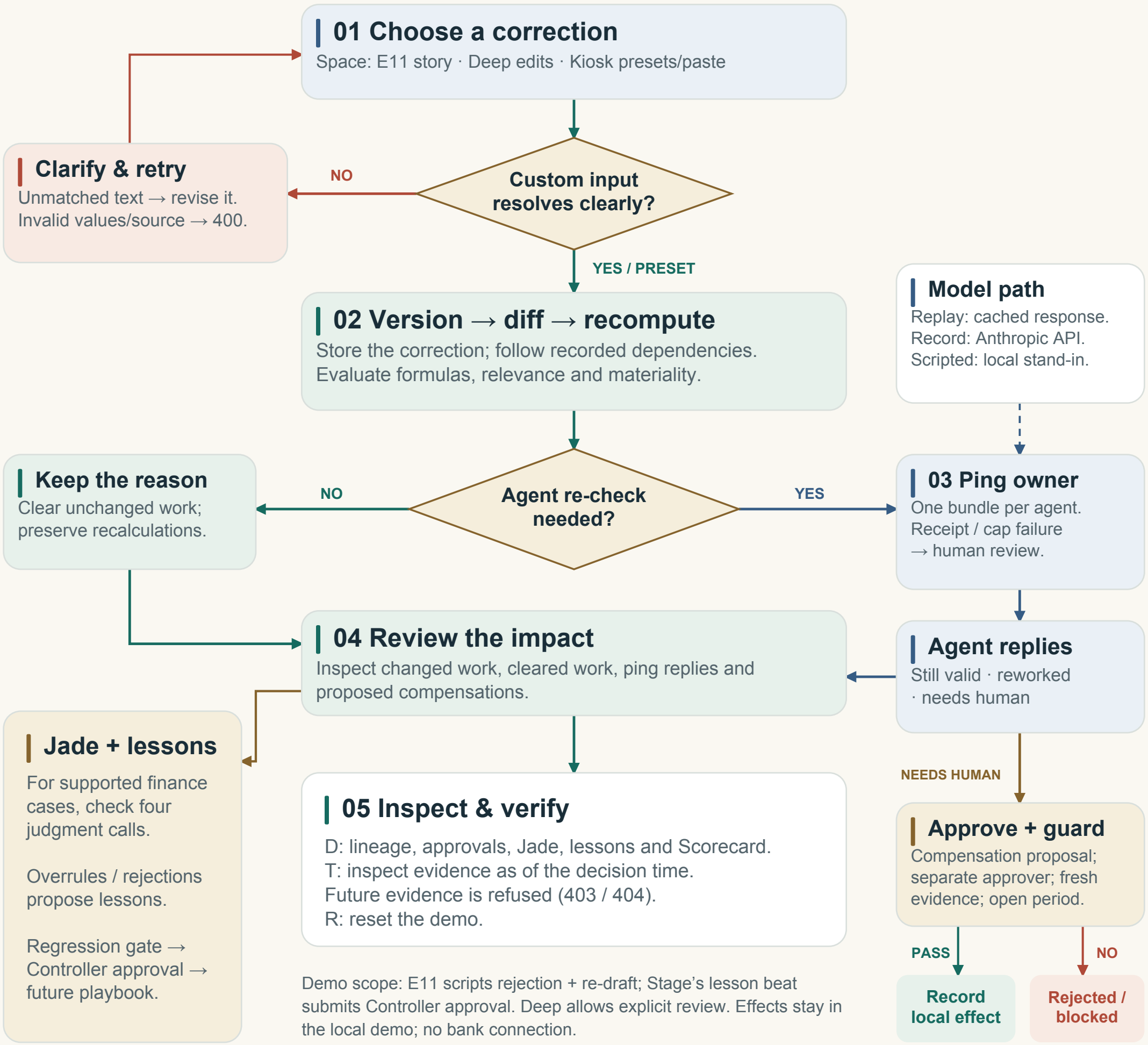
03 / COMPLETE USER JOURNEY

Follow the correction.

One main path, with explicit stops for uncertain input, agent review and human control.
The Stage walkthrough and Deep workspace expose the same backend.

USER · SYSTEM · DECISION / GUARD

Solid arrows = flow · Dashed = optional model path



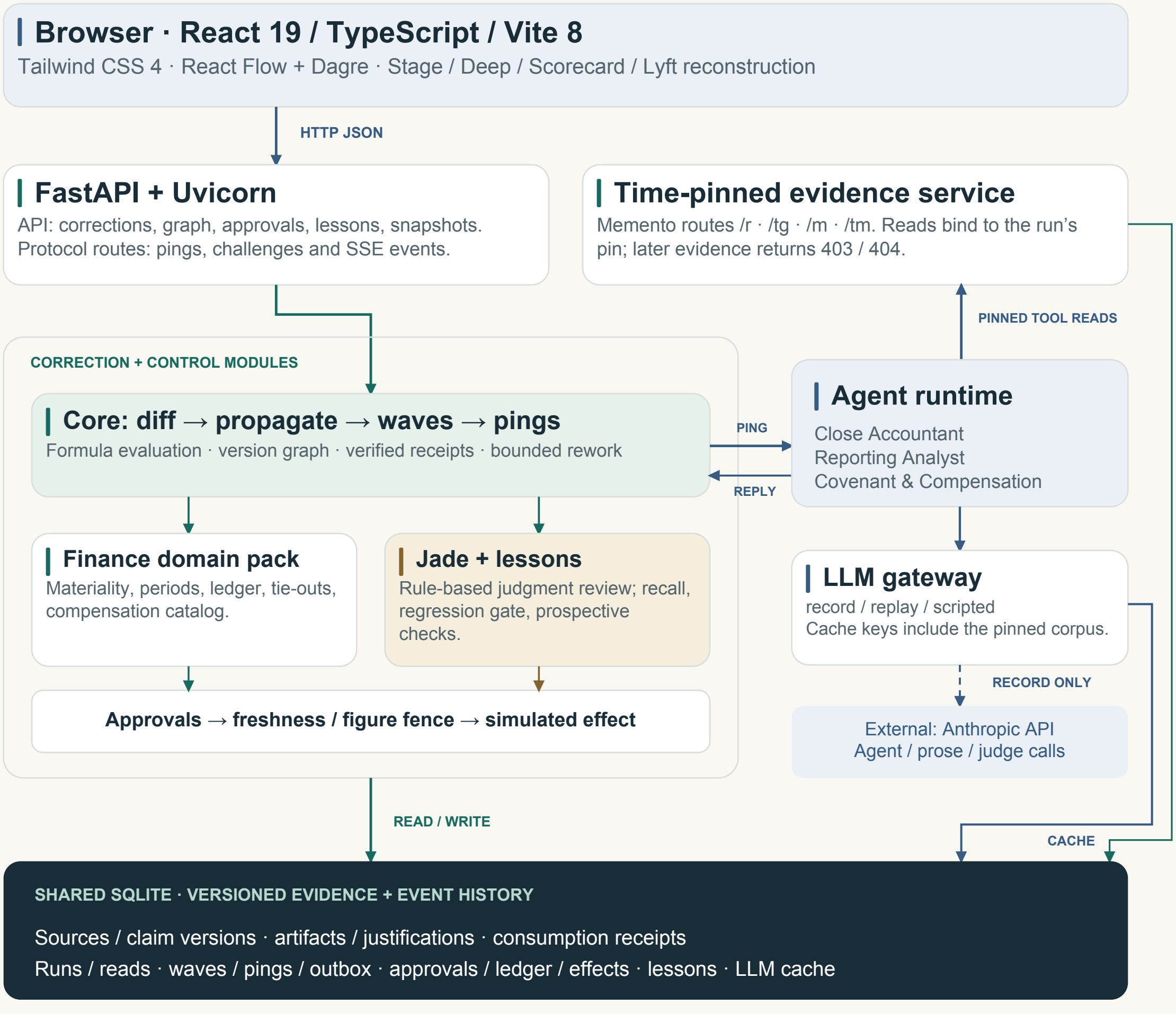
Demo scope: E11 scripts rejection + re-draft; Stage's lesson beat submits Controller approval. Deep allows explicit review. Effects stay in the local demo; no bank connection.

YOUR REVIEW Trace one successful correction. Circle where you would stop for more evidence.

04 / TECHNICAL ARCHITECTURE

A graph with memory and guardrails.

One Python service hosts the demo API, time-pinned documents and built frontend.
Replay uses cached or local responses; external model calls require record mode.



Offline builders restore the canonical snapshot; evaluation scripts compare against YAML answer keys.
Also implemented: stdio MCP read tools, Docker/Fly configuration and a Vercel entry with a /tmp database.
MCP/A2A notification adapters and a second domain pack remain future work.

YOUR REVIEW Mark the first boundary to change for multiple concurrent customer sessions.

EVIDENCE Service entry · Core / pack boundary · MCP read tools · Vercel entry

05 / DEVELOPMENT JOURNEY & LESSONS

From a working graph to a usable demo.

Actual Git milestones, shown in Pacific time. Engineering lessons below come from the decision log and source tests; they are not a reconstructed hour-by-hour diary.

THE BUILD RECORD

23 OCT · 22:57 06bbbc9

Offline demo + runtime fixes

Snapshot restore via SQLite backup; proxies and verification tools arrive in the working demo.

24 OCT · 11:27 4b6031c

Jade, lessons and explanations

Add judgment review, lesson gating and recall; expose the learning sequence and hover explanations.

24 OCT · 11:46 fcc8cb3

Make the interface teach itself

Opt-in kiosk, plain-language counters and a first-visit guide explain the Deep workspace.

24 OCT · 12:11 48964da

Keep the kiosk loop running

Synthetic mouse movement no longer interrupts the attract loop; later screenshots document the check.

24 OCT · 15:34 e452169

Simplify the story; package hosting

Scorecard naming, clearer dependency lines and a Vercel entry with committed frontend assets.

STORED EVALUATION EVIDENCE

0 / 288

future-data leaks

Adversarial pinned fetches.

162 / 162

identical replay artifacts

Byte-for-byte reconstruction.

13 / 13

planted mistakes caught

Jade: 0 false overrules on 63 correct claims.

Reported by committed replay evaluations, not rerun for this notebook. These measure the seeded demo cases, not production reliability.

WHAT THE ENGINEERING TAUGHT US

Reset without stale reads

Replacing a live DB left stale handles. SQLite backup preserves readers; concurrent-reset tests cover it.

Match build and replay

Scripted and replay prose diverged. Both now reuse the same recorded response when the cache key matches.

Explain the graph

Stage shows one beat at a time; Deep exposes the evidence. Layout sizes and a type-floor check improved legibility.

NOW Correction tracing, agent replies, approvals, time travel, Jade and gated lessons work in the offline demo.

NEXT Record the remaining scripted rework; wire prose critique; isolate sessions; add notification adapters. Current limits: mixed cache origins, code-based Jade, one shared world per instance.

YOUR REVIEW Team reflection: next experiment / owner / evidence of success / review date.